Intrusion Detection sul dataset UNSW-NB15

Classificazione Binaria e Multiclasse del Traffico di Rete

Simone Montagnini

Università di Bologna

Settembre 2026

Indice

1. Introduzione
2. Analisi Esplorativa dei Dati
3. Preprocessing
4. Selezione degli attributi
5. Classificazione
6. Conclusioni

Obiettivi del Progetto

Questo progetto affronta il rilevamento delle intrusioni nel traffico di rete attraverso due task di classificazione:

- **Classificazione binaria**: distinzione tra traffico normale e malevolo (`label`)
- **Classificazione multiclasse**: identificazione della minaccia specifica tra 9 possibili tipologie (`attack_cat`)

Sorgente

Competizione Kaggle: *Advanced AI for Cybersecurity — Intrusion Detection with UNSW-NB15*

II Dataset — UNSW-NB15

- Dataset di traffico di rete realistico contenente attività sia normali che malevole
- **175.341** record di training, **36** feature iniziali (categoriche e numeriche)
- Due colonne target:
  - `label`: 0 = normale, 1 = attacco
  - `attack_cat`: Normal, Generic, Exploits, Fuzzers, DoS, Reconnaissance, Analysis, Backdoor, Shellcode, Worms

Distribuzione del Target

Binaria (`label`)

- Attacco: 68.1%
- Normale: 31.9%

Multiclasse (`attack_cat`)

- Normal: 56.000
- Generic: 40.000
- Exploits: 33.393
- Fuzzers: 18.184
- DoS: 12.264
- Reconnaissance: 10.491
- Analysis: 2.000
- Backdoor: 1.746
- Shellcode: 1.133
- **Worms: 130**

Sbilanciamento

Il target binario è relativamente bilanciato, ma `attack_cat` presenta forti sbilanciamenti: la classe Worms è **430 volte meno numerosa** rispetto a Normal; da qui la necessità di strategie di **class-weighting** nel modello multiclasse.

Qualità dei Dati

- **Righe duplicate**: 78.519 su 175.341 (**44,8%**)
  - Rimosse prima dello split, così da evitare data leakage tra train e test
  - Impatto sulla distribuzione: il task binario diventa quasi bilanciato (**50,5% Normal /49,5% Attack**)

- **Feature categoriche**:
  - `proto` (Alta cardinalità): 133 valori unici (`tcp`/`udp` $\approx$ 82% del traffico)
  - `service`: 13 valori, 54% mancanti/sconosciuti (-)
  - `state`: 9 valori

Feature Numeriche Asimmetriche

Diverse feature del traffico presentano distribuzioni right-skewed:

Feature	**Asimmetria**
trans_depth	167.3
response_body_len	76.3
sbytes	45.3
sloss	44.8
dloss	41.4
spkts	40.2

27 feature superano la soglia (> 1.0) nell'EDA sul dataset grezzo; dopo la deduplicazione sul training set, **27 feature** sono risultate asimmetriche e trasformate con `log1p`.

Preparazione & Trasformazione Dati

1. **Rimozione duplicati**: rimosse 78.519 righe duplicate prima dello split
2. **Split train/test**: 67/33, stratificato su `attack_cat` per preservare le proporzioni di tutte le classi in entrambi i task
3. **Encoding**: `OneHotEncoder` su `proto`, `service`, `state` — **184 feature totali**
4. **Correzione asimmetria**: trasformazione `log1p` su 27 feature (asimmetria > 1.0)

Prevenzione Data Leakage

Scelta chiave

Le operazioni di preprocessing eseguono il **fit solamente sul training set** e vengono applicate sul test set grazie a `transform()` così da evitare data leakage.

- **Netta separazione**: Fare il fit degli encoder sull'intero dataset inietta conoscenza futura nella fase di addestramento
- **Gestione dati mai visti**: Categorie rare (e.g. all'interno dei 133 valori di `proto`) potrebbero apparire solamente nel test set
- **Tolleranza ai guasti**: `OneHotEncoder` è configurato con `handle_unknown='ignore'` per aggirare nuove categorie senza causare blocchi durante l'esecuzione

SelectKBest

- **Feature selection supervisionata**: ogni feature viene valutata rispetto al target, mantenendo solamente le migliori k feature
- **Funzione di score**: ANOVA F-value (`f_classif`)
  - `mutual_info_classif` è stata esclusa a causa del costo computazionale eccessivo data la dimensione del dataset.
- **Selezione**: $k = 50$ su 184 feature; integrata nella `Pipeline` con tuning di $k \in \{30, 40, 50\}$ in Cross-Validation

Preservazione dell'Interpretabilità

SelectKBest mantiene gli attributi originali del traffico. In questo modo il modello rimane trasparente, permettendo di ricondurre direttamente le attività malevole a specifici segnali di rete.

PCA

- **Scaling preliminare**: feature standardizzate (`StandardScaler`), con fit della PCA esclusivamente sul training set
- **Varianza spiegata**: mantenere il 90% della varianza richiede **136 componenti** (118 per l'80%)
- **Riduzione limitata**: dovuta alle numerose colonne generate dal `OneHotEncoder`: una volta standardizzate, queste feature risultano quasi incorrelate tra loro e non possono essere compresse efficacemente

Analisi delle Componenti e Contributo delle Feature

Piuttosto che essere dominate dalle numerose colonne `proto` codificate con one-hot encoding, le componenti principali mostrano un mix di tipologie di dati sottostanti. Per esempio, la PC2 è guidata quasi interamente da feature legate alle tempistiche e ai byte (`sinpkt`, `dload`, `tcprtt`)

Confronto sulla Selezione delle Feature

	SelectKBest	**PCA**
Dimensionalità	30–50 feature	136 componenti
Interpretabilità	Alta (dominio originale)	Bassa (combinazioni lineari)
Trasformazione	Supervisionata (target-aware)	Non supervisionata
Efficienza di training	Accettabile	Proibitiva

Scelta di Progettazione

PCA è stata esclusa dalla pipeline di classificazione: anche con un numero ridotto di componenti, l'overhead computazionale durante la `GridSearchCV` è risultato proibitivo. Pertanto, la classificazione procede solamente con SelectKBest.

Metodologia di Classificazione

- **Pipeline**: Decision Tree e Random Forest inseriti in una `Pipeline` (`VarianceThreshold` + `SelectKBest` + `Classifier`), confrontati con Random Forest di **baseline**
- **Tuning degli Iperparametri**: `selector__k` $(30, 40, 50)$, `max_depth` e `criterion` (DT), `n_estimators` e `max_depth` (RF)
- **Gestione dello Sbilanciamento**: incluso `class_weight` (`None` vs `'balanced'`) per valutare la strategia migliore sui vettori di minaccia rari
- **Validazione**: 3-fold `StratifiedKFold` sul training set; test set usato solo per la valutazione finale

Metriche di Valutazione

L'Accuracy è fuorviante su dataset fortemente sbilanciati. I modelli sono valutati sulla base di **F1-Score** (classificazione binaria) e **Macro F1-Score** (classificazione multiclasse) per pesare correttamente le performance sulle classi minoritarie.

Risultati Classificazione Binaria

Performance in Cross-Validation

Metrica	RF Base	DT	RF Best
Accuracy	–	0.913	0.919
Precision	–	0.879	0.885
Recall	–	0.956	0.963
F1-Score	**0.919**	**0.916**	**0.922**

- **Test Set (RF, $k = 50$): F1 = 0.921**, Recall attacchi **97%**, False Positive Rate **13%** sul traffico normale.
- **Effetto Class Weight**: a parità di parametri ottimali, `None` (0.922) e `'balanced'` (0.921) sono equivalenti (target già bilanciato post-deduplicazione).

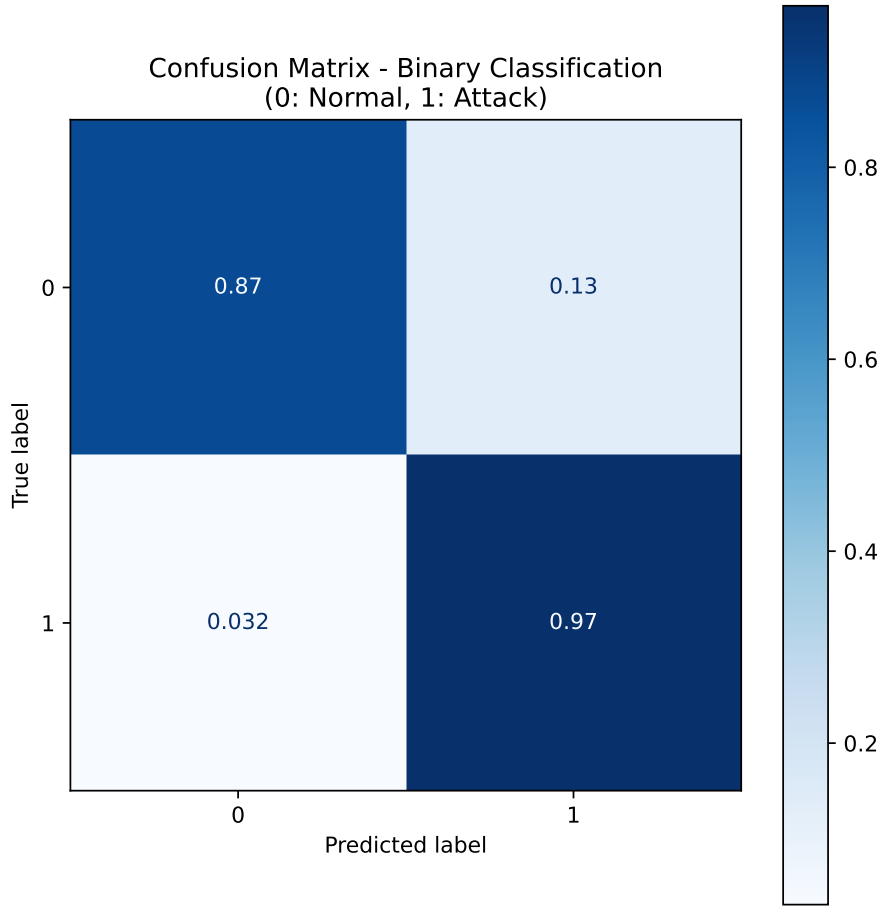


Figura: Matrice di Confusione Binaria

Risultati Classificazione Multiclasse

La sfida Multiclasse

- **Random Forest ($k = 40$)**: raggiunge un **Macro F1 di 0.561** sul test set (**0.559** in CV vs **0.540** di DT e **0.461** della Baseline a 184 feature)
- **Classi maggioritarie** (Normal, Exploits) predette con alta precisione
- **Classi minoritarie**: alta confusione e bassa precision (es. Analysis 0.17, Backdoor 0.16, Shellcode 0.26)

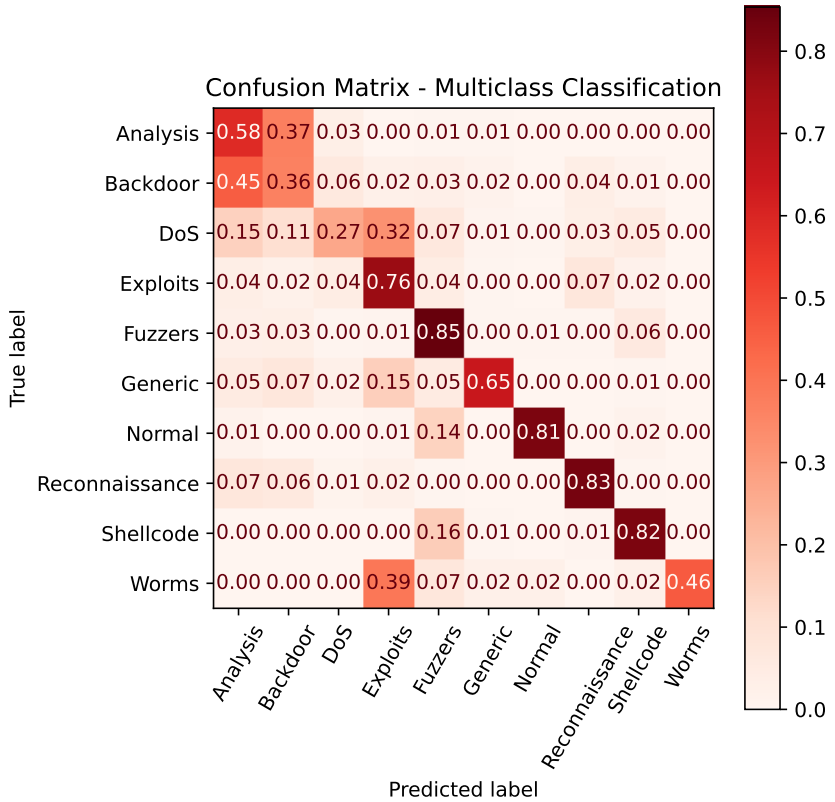


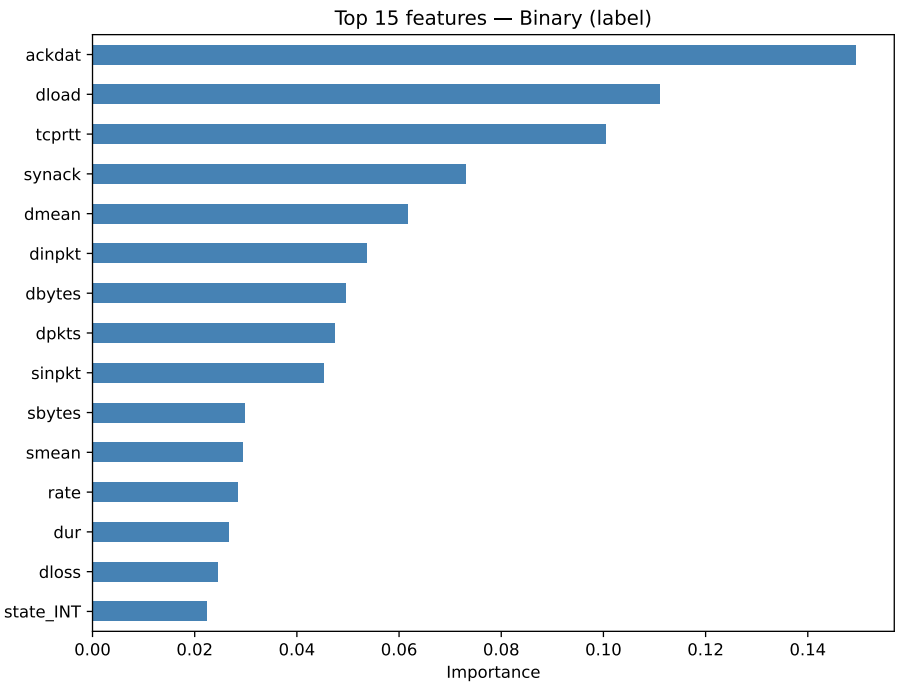
Figura: Matrice di Confusione Multiclasse

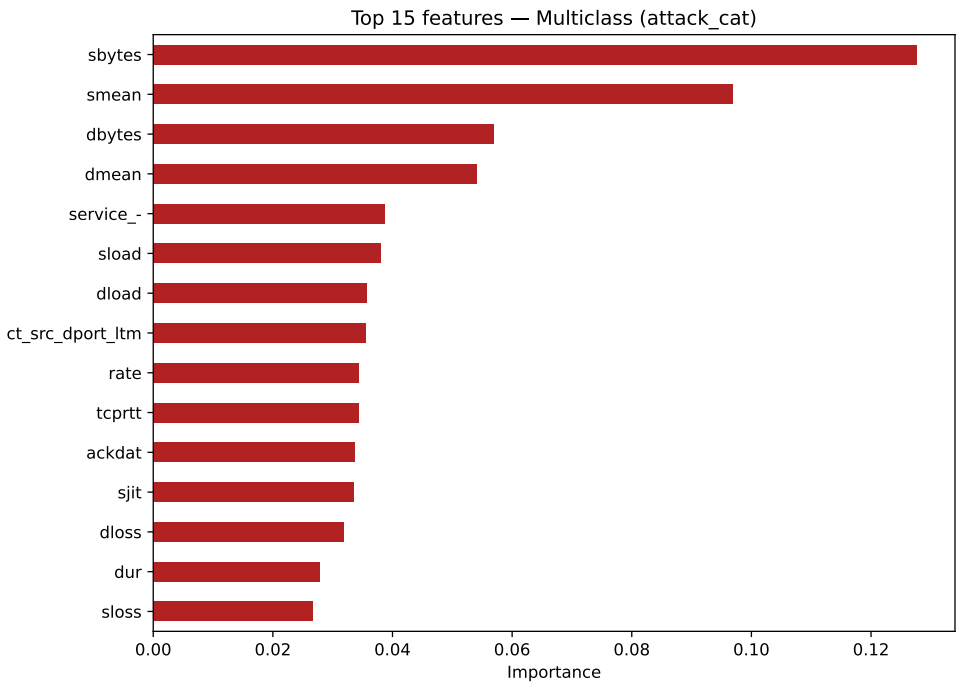
Impatto del Bilanciamento

Grazie a `class_weight='balanced'`, il modello riesce a rilevare anche istanze di attacchi molto rari (come Worms), evitando quindi che vengano completamente ignorate.

Analisi Feature Importance

- **Task binario**: si basa prevalentemente sulle feature temporali del TCP handshake (`ackdat`, `synack`, `tcprtt`): gli handshake anomali rappresentano un forte indicatore di traffico malevolo
- **Task multiclasse**: si affida a feature volumetriche legate al trasferimento di byte (`sbytes`, `smean`, `dbytes`) per distinguere il *tipo* di attacco





Valutazione Progetto

- **Performance e Trade-off Operativi**: Il modello binario ottiene un ottimo F1-Score (**0.921**, recall attacchi 97%), ma il **13% di falsi positivi** sul traffico normale genererebbe molti falsi allarmi in scenari reali fortemente sbilanciati verso il traffico benigno
- **Il limite del Multiclasse**: Distinguere le singole categorie resta complesso (**Macro F1: 0.561**), con forti sovrapposizioni tra classi affini (Backdoor/Analysis, DoS/Exploits) e tra Fuzzers e traffico Normal (13%)
- **Robustezza alla Multicollinearità**: La rimozione manuale di 10 feature altamente correlate ($r \geq 0.9$) ha ridotto leggermente il Macro F1 in CV (da **0.559** $\pm$ **0.001** a **0.552** $\pm$ **0.002**), confermando che i modelli ad albero gestiscono nativamente la ridondanza

Sviluppi Futuri

Per superare i limiti attuali e portare il sistema verso un Network Intrusion Detection System (NIDS) realistico, iterazioni future potrebbero esplorare:

- **Resampling Avanzato**: nonostante SMOTE sia stato inizialmente valutato, è stato scartato a causa dell'estremo overhead computazionale. Lavori futuri dovranno indagare tecniche di campionamento altamente ottimizzate e scalabili
- **Architetture Alternative**: Valutare classificatori non basati su alberi per determinare se confini matematici differenti possano isolare meglio classi rare
- **Feature Engineering di Dominio**: derivare nuove feature di rete altamente specifiche, mirate a distinguere meglio la sovrapposizione delle firme degli attacchi minoritari rispetto alle anomalie più frequenti